SECURITY OPERATIONS CENTER (SOC) PROJECT REPORT

Sysmon Integration with Wazuh

Advanced Endpoint System Monitoring through SIEM Integration

Prepared by: Muhammad Khubab

Domain: SIEM Administration & Endpoint Threat Detection

Tools Used: Wazuh SIEM, Sysmon, Windows Endpoint

linkedin.com/in/muhammad-khubab-475046204

github.com/mkhubab11

October 2026

Cybersecurity / SOC Hom Lab Project

Table of Contents

1. Introduction 3
    1.1 Objectives 3
    1.2 Scope 3
2. Tools and Environment 3
3. Implementation 3
    3.1 Installing Sysmon on the Windows Endpoint 4
    3.2 Applying a Sysmon Configuration 4
    3.3 Verifying Sysmon Installation 5
    3.4 Wazuh Server and Agent Deployment 5
    3.5 Configuring the Wazuh Agent to Collect Sysmon Logs 6
4. Detection Testing and Results 6
    4.1 Simulating Suspicious Activity 6
    4.2 Viewing Alerts in the Wazuh Dashboard 7
    4.3 MITRE ATT&CK Mapping 7
    4.4 Detailed Alert Analysis 7
5. Key Findings 9
6. Conclusion 9

1. Introduction

Windows' built-in event logging provides limited visibility into process activity, making it difficult to detect advanced techniques such as fileless execution, process injection, or script-based attacks. Sysmon (System Monitor), part of the Microsoft Sysinternals suite, extends this visibility by logging detailed information about process creation, network connections, file creation, and other system behavior directly to the Windows Event Log. This project documents the integration of Sysmon with Wazuh, forwarding Sysmon's rich telemetry into the SIEM so that endpoint activity can be centrally collected, correlated, and mapped to adversary techniques using the MITRE ATT&CK framework.

1.1 Objectives

- Install and configure Sysmon on a Windows endpoint using a community-maintained, modular configuration.
- Verify that Sysmon is generating detailed telemetry in the Windows Event Log.
- Configure the Wazuh agent to collect Sysmon's Operational event channel.
- Simulate suspicious endpoint activity to validate end-to-end detection.
- Analyze the resulting alerts and their mapping to MITRE ATT&CK techniques in the Wazuh dashboard.

1.2 Scope

The project is implemented in a lab environment consisting of a Wazuh manager/dashboard node and a Windows endpoint (window_agent) running both the Wazuh agent and Sysmon. Detection is demonstrated using everyday endpoint activity browser-driven script file creation and a deliberately crafted fileless PowerShell command that Sysmon's rule set is designed to surface. The scope covers log collection and alert analysis; it does not include automated response or blocking actions.

2. Tools and Environment

Component	Details
SIEM Platform	Wazuh (Manager + Dashboard)
Endpoint Telemetry Source	Sysmon v15.22 (Sysinternals)
Sysmon Configuration	sysmon-modular (olafhartong), balanced profile
Monitored Endpoint	Windows 11 (agent: window_agent)
Log Channel	Microsoft-Windows-Sysmon/Operational
Configuration File	agent.conf (window_Lab group, Wazuh Manager)

3. Implementation

3.1 Installing Sysmon on the Windows Endpoint

Sysmon is downloaded directly from the official Microsoft Sysinternals site, which distributes the executable along with its Windows and Linux (GitHub) variants.

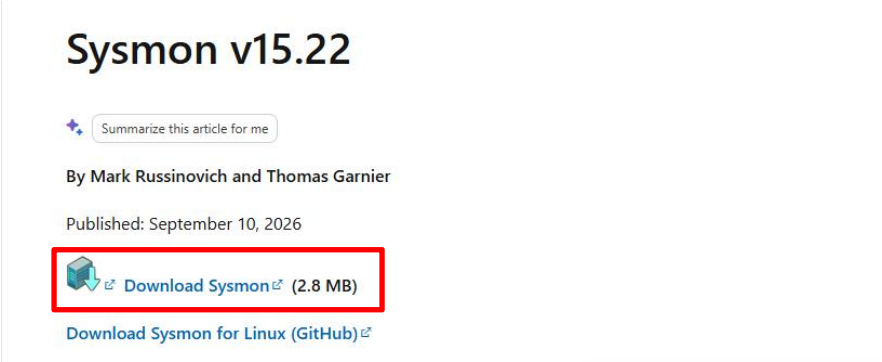
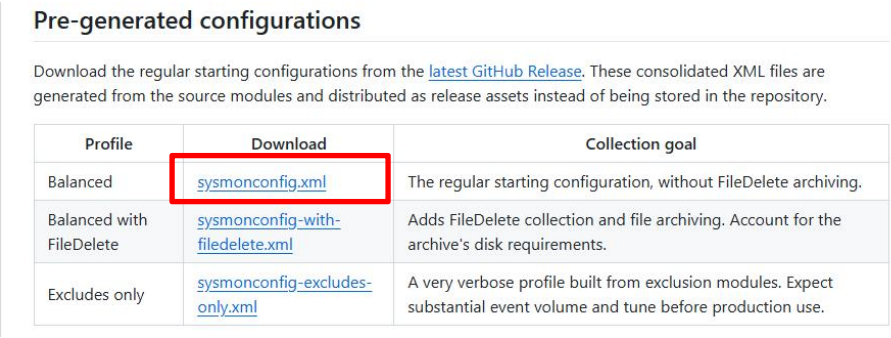


Figure 1: Sysmon v15.22 download page on the official Microsoft Sysinternals site.

After downloading, the archive is extracted to reveal the Sysmon executables (Sysmon.exe, Sysmon64.exe, Sysmon64a.exe) and the End User License Agreement.

3.2 Applying a Sysmon Configuration

A Sysmon configuration file is an XML file that defines which system events Sysmon should monitor and log, and how to filter or exclude noisy or irrelevant events. Rather than writing a configuration from scratch, this project uses the community-maintained sysmon-modular project, which publishes pre-generated, regularly updated configurations as GitHub release assets.



Pre-generated configurations

Download the regular starting configurations from the latest GitHub Release. These consolidated XML files are generated from the source modules and distributed as release assets instead of being stored in the repository.

Profile	Download	Collection goal
Balanced	sysmonconfig.xml	The regular starting configuration, without FileDelete archiving.
Balanced with FileDelete	sysmonconfig-with-filedelete.xml	Adds FileDelete collection and file archiving. Account for the archive's disk requirements.
Excludes only	sysmonconfig-excludes-only.xml	A very verbose profile built from exclusion modules. Expect substantial event volume and tune before production use.

Figure 2: Pre-generated Sysmon configuration profiles from the sysmon-modular GitHub repository; the "Balanced" profile was selected.

The downloaded sysmonconfig.xml file is placed in the same folder as the Sysmon executables:

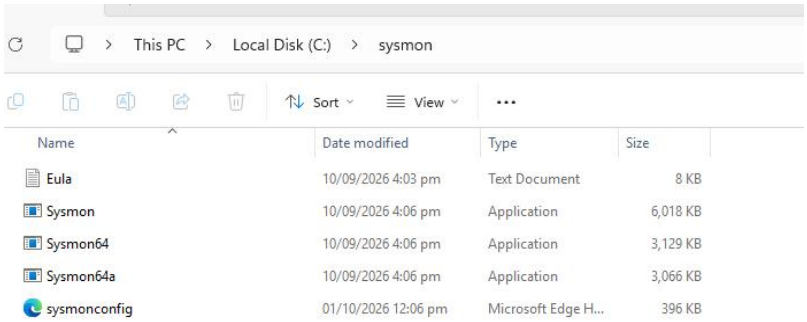


Figure 3: Sysmon installation folder (C:\sysmon) containing the executables and the sysmonconfig.xml configuration file.

Sysmon is then installed with the configuration applied, run from an elevated PowerShell session:

```
.\Sysmon64.exe -accepteula -i sysmonconfig.xml
```

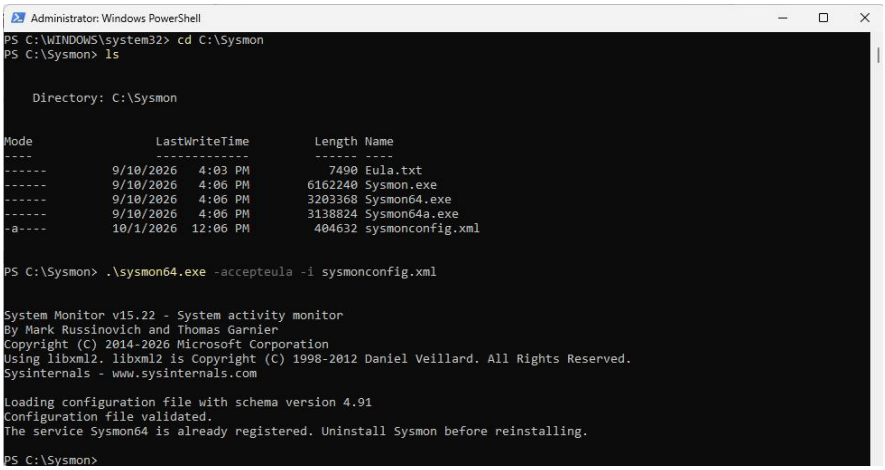


Figure 4: Sysmon64 installed successfully with the sysmonconfig.xml configuration file, schema version 4.91.

3.3 Verifying Sysmon Installation

To confirm Sysmon is installed correctly and actively generating events, the Windows Event Viewer is used to navigate to: Application and Service Logs → Microsoft → Windows → Sysmon → Operational. A correctly configured installation shows a continuous stream of process, network, and file events in this log.

3.4 Wazuh Server and Agent Deployment

A Wazuh manager and dashboard are deployed to serve as the central monitoring platform, and a Wazuh agent is installed on the same Windows endpoint running Sysmon so that its telemetry can be collected and forwarded for analysis.

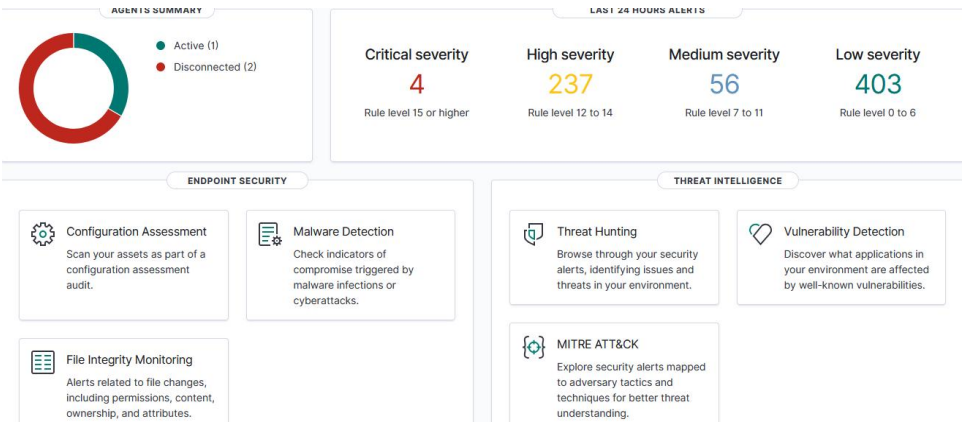


Figure 5: Wazuh dashboard overview after deployment, showing the connected agent and baseline alert severity distribution.

3.5 Configuring the Wazuh Agent to Collect Sysmon Logs

The Wazuh agent configuration for the endpoint's group (window_Lab) is edited to add a localfile block that reads Sysmon's Operational event channel using the eventchannel log format:

```
<agent_config os="window">

  <!--Sysmon log Collection-->
  <localfile>
    <location>Microsoft-Windows-Sysmon/Operational</location>
    <log_format>eventchannel</log_format>
  </localfile>
</agent_config>
```

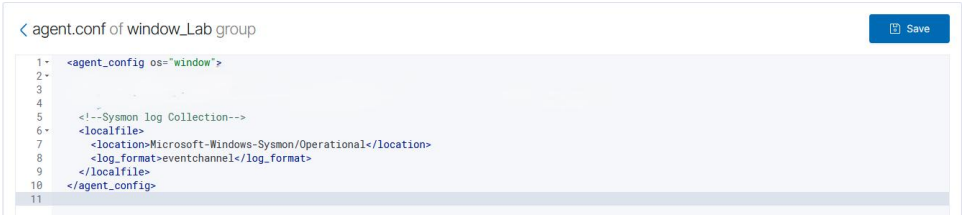


Figure 6: Sysmon log-collection block added to the agent.conf for the window_Lab group in the Wazuh manager.

The Wazuh manager is restarted to apply the updated group configuration: sudo systemctl restart wazuh-manager

4. Detection Testing and Results

4.1 Simulating Suspicious Activity

To validate the pipeline, a fileless-execution style command is run on the endpoint, launching PowerShell in hidden, no-profile mode to execute an inline command — a pattern commonly associated with living-off-the-land and fileless malware techniques:

```
powershell.exe -nop -w hidden -c "Write-Host 'Testing Fileless Detection'"
```

```
PS C:\Sysmon> powershell.exe -nop -w hidden -c "Write-Host 'Testing Fileless Detection'"
```

Figure 7: Fileless-execution test command run on the Windows endpoint to validate Sysmon detection.

4.2 Viewing Alerts in the Wazuh Dashboard

Shortly after the test, corresponding alerts appear in the Wazuh dashboard. In addition to the simulated test, Sysmon's modular ruleset also picked up everyday browser activity — scripting files being written under user/temp folders — along with process and logon events, demonstrating broad visibility beyond the single test case.

432 hits

Sep 30, 2026 @ 15:52:39.687 - Oct 1, 2026 @ 15:52:39.688

Export Formatted | 862 available fields | Columns | Density | 1 fields sorted | Full screen

Inspect document details

	agent.name	rule.description	rule.level	rule.id
Oct 1, 2026 @ 15:51:11.251	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:51:11.219	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:51:08.656	window_agent	Process loaded taskschd.dll module. May be used to create delayed malware execution	4	92154
Oct 1, 2026 @ 15:51:02.620	window_agent	Windows Logon Success	3	60106
Oct 1, 2026 @ 15:50:59.986	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:50:59.900	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:50:55.050	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:50:55.000	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:50:50.333	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:50:50.304	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:50:49.221	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:50:49.220	window_agent	Scripting file created under Windows Temp or User folder	6	92200
Oct 1, 2026 @ 15:50:49.158	window_agent	Scripting file created under Windows Temp or User folder	6	92200

Figure 8: Wazuh alerts generated from Sysmon telemetry, including scripting-file creation events and a suspicious module load (taskschd.dll).

4.3 MITRE ATT&CK Mapping

The Wazuh MITRE ATT&CK module aggregates alerts for the endpoint and maps them to adversary tactics and techniques, giving a tactic-level view of the activity observed over the monitoring window.

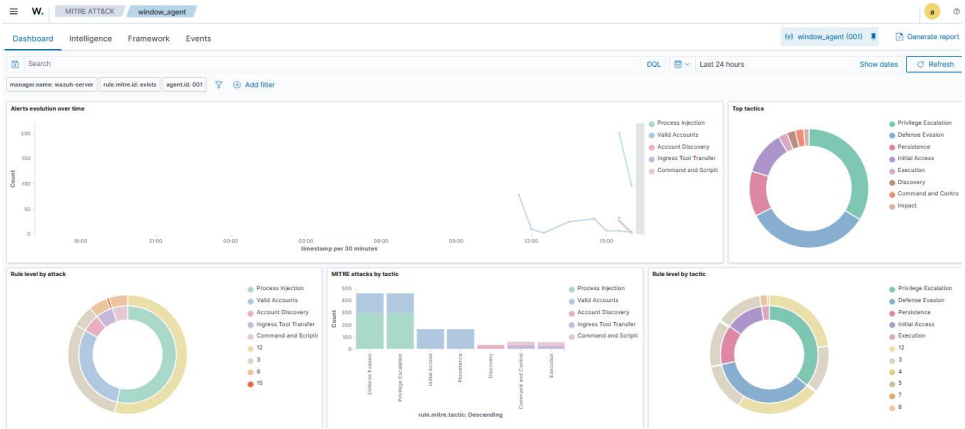


Figure 9: MITRE ATT&CK dashboard for window_agent, showing alerts distributed across tactics including Defense Evasion, Privilege Escalation, Discovery, and Command and Control.

4.4 Detailed Alert Analysis

An individual alert is opened to inspect the full event detail. In this example, Sysmon captured Firefox writing a JavaScript file to a user profile folder, which the rule set classified under technique T1059.007 (Command and Scripting Interpreter: JavaScript).

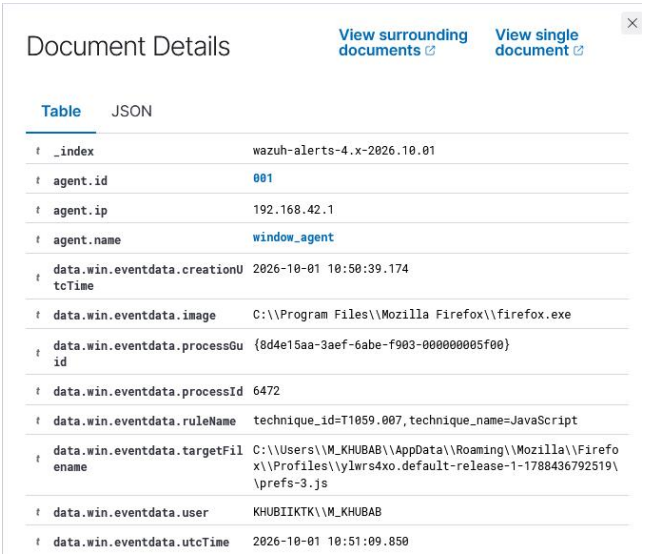


Figure 10: Alert detail showing the process (firefox.exe), target file path, and MITRE technique mapping (T1059.007).

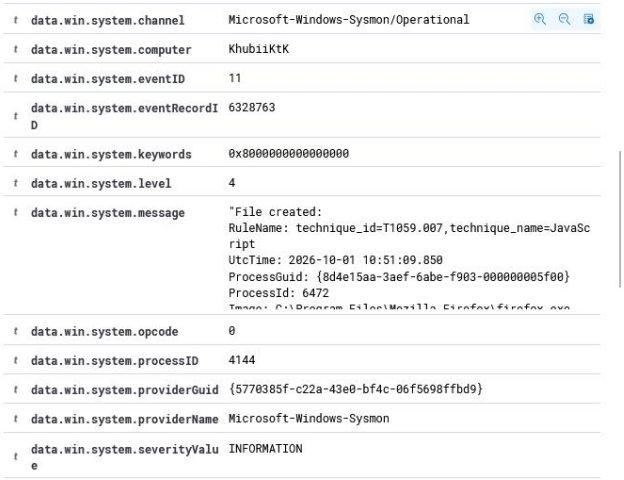


Figure 11: Underlying Sysmon event data, including the raw event message, event ID 11 (FileCreate), and provider details.

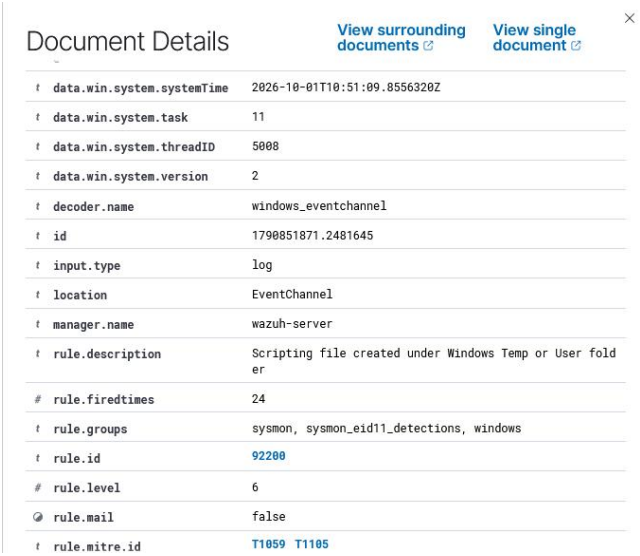


Figure 12: Final rule metadata showing the alert classified under the sysmon / sysmon_eid11_detections rule groups, with MITRE IDs T1059 and T1105.

5. Key Findings

- Sysmon, combined with the sysmon-modular configuration, provided significantly richer telemetry than default Windows auditing, capturing process creation, file creation, and module-load events in real time.
- The Wazuh agent successfully ingested Sysmon's Operational event channel via the eventchannel log format with no additional parsing required.
- Sysmon's modular ruleset flagged routine but notable behavior — scripting files created under Temp/User folders, and a process loading taskschd.dll (often abused for persistence via scheduled tasks) — in addition to the deliberately simulated fileless PowerShell execution.
- Alerts were automatically enriched with MITRE ATT&CK technique and tactic IDs (e.g., T1059.007, T1105), enabling tactic-level trend analysis directly from the Wazuh MITRE ATT&CK module.
- Full event context — process GUID, parent process, target file path, and the affected user account — was preserved in each alert, giving analysts enough detail to triage without needing to return to the endpoint.

6. Conclusion

Integrating Sysmon with Wazuh substantially improves endpoint visibility beyond what native Windows Event Logs provide. Sysmon's granular, rule-driven telemetry gives Wazuh the raw detail needed to detect suspicious process behavior, scripting activity, and technique-level adversary patterns, while Wazuh's MITRE ATT&CK mapping turns that telemetry into an actionable, tactic-oriented view for analysts. The testing performed confirmed that the pipeline reliably captures both deliberately simulated attack behavior and naturally occurring suspicious activity on the endpoint.

Overall, this project demonstrates how a lightweight, well-configured sysinternals tool can meaningfully close the visibility gap between basic SIEM log collection and true endpoint detection and response (EDR)-style monitoring.